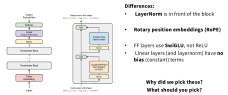
Advanced Artificial Intelligence and Machine Learning

Transformers Modernized



Learning goals

- What changed in the Transformer block since 2017 and why: pre-norm, RMSNorm, no biases, gated activations
- Rotary position embeddings (RoPE)
- The consensus hyperparameters of modern language models
- Stability tricks and attention variants (MQA/GQA, sliding windows), and why memory matters
- Mixture of experts: routing, training, DeepSeek-MoE

Since 2017: What Changed Inside the Transformer?

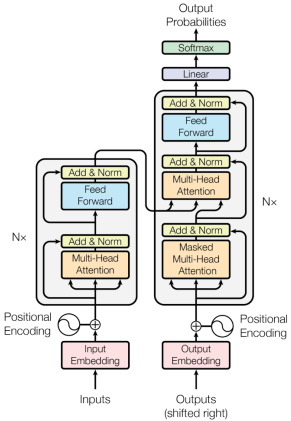
SINCE 2017: WHAT CHANGED INSIDE THE TRANSFORMER?

- Session 7 introduced the original 2017 block: post-norm with LayerNorm, sinusoidal position embeddings, a ReLU feed-forward network. GPT, BERT and T5 (session 8) already changed some of these details, e.g. learned or relative position embeddings.
- Today's large models (e.g. the LLaMA family, DeepSeek) keep the overall design, but change several components *inside* the block.
- This session goes through these changes and the evidence behind them: normalization, activations, position embeddings, hyperparameters, stability, attention heads.
- Then a change at the level of the whole layer: **mixture of experts**.
- Source: Stanford CS336, Lectures 3 and 4 (Spring 2025).

OUTLINE AND GOALS

- Quick recap of the 'standard' transformer
  - What do most of the large LMs have in common?
- What are common variations to the architecture / training process?
  - Today's theme: the best way to learn is hands-on experience
  - the second best way is to try to learn from others' experience

STARTING POINT: THE 'ORIGINAL' TRANSFORMER



Review: choices in the standard transformer

Position embedding: sines and cosines

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$

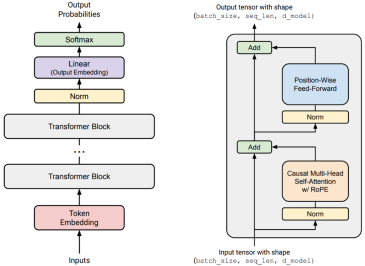
$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

FFN: ReLU

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Norm type: post-norm, LayerNorm

A SIMPLE, MODERN VARIANT



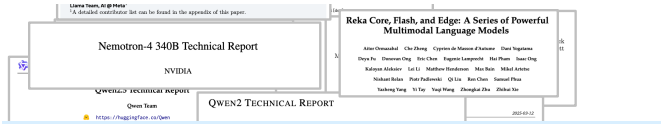
Differences:

- **LayerNorm** is in front of the block
- **Rotary position embeddings (RoPE)**
- FF layers use **SwiGLU**, not ReLU
- Linear layers (and layernorm) have **no bias** (constant) terms

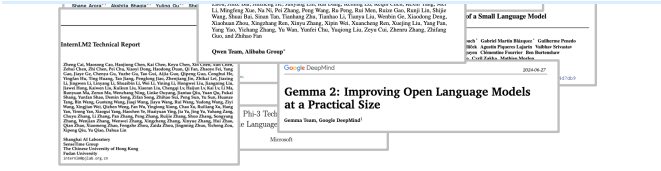
Why did we pick these?
What should you pick?

HOW SHOULD WE THINK ABOUT ARCHITECTURES?

Lots of architecture. Just in the last year since last 336..

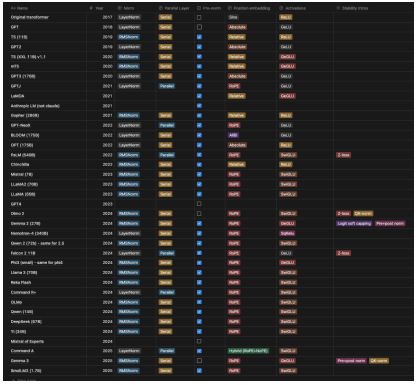


Over 19 new *dense* model releases, many of them with minor architecture tweaks..



LET'S LOOK AT THE DATA (ON DENSE ARCHITECTURES)

Learn from the many other models (and papers) out there



Name	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations	Stability tricks
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU	
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU	
T5 (11B)	2019	RMSNorm	Serial	■	Relative	ReLU	
GPT-3	2019	LayerNorm	Serial	■	Absolute	GeLU	
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	■	Relative	GeGLU	
mT5	2020	RMSNorm	Serial	■	Relative	GeGLU	
GPT-3 (175B)	2020	LayerNorm	Serial	■	Absolute	GeLU	
GPT-J	2021	LayerNorm	Parallel	■	RoPE	GeLU	
LaMDA	2021			■	Relative	GeGLU	
Anthropic LM (not claude)	2021			■			
Gopher (280B)	2021	RMSNorm	Serial	■	Relative	ReLU	
GPT-Neox	2022	LayerNorm	Parallel	■	RoPE	GeLU	
BLOOM (176B)	2022	LayerNorm	Serial	■	ABB	GeLU	
OPT (175B)	2022	LayerNorm	Serial	■	Absolute	ReLU	
PaLM (540B)	2022	RMSNorm	Parallel	■	RoPE	SwiGLU	Z-loss
Chinchilla	2022	RMSNorm	Serial	■	Relative	ReLU	
Mistral (7B)	2023	RMSNorm	Serial	■	RoPE	SwiGLU	
LLaMA2 (70B)	2023	RMSNorm	Serial	■	RoPE	SwiGLU	
LLaMA (65B)	2023	RMSNorm	Serial	■	RoPE	SwiGLU	
GPT-4	2023			☐			
Olmo 2	2024	RMSNorm	Serial	☐	RoPE	SwiGLU	Z-loss, QK-layer
Gemma 2 (27B)	2024	RMSNorm	Serial	■	RoPE	GeGLU	Logit soft-capping, Pre+post norm
Nemotron-4 (340B)	2024	LayerNorm	Serial	■	RoPE	SqReLU	
Qwen 2 (72B) - same for 2.5	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Falcon 2 11B	2024	LayerNorm	Parallel	■	RoPE	GeLU	Z-loss
Phi3 (small) - same for (phi)	2024	RMSNorm	Serial	■	RoPE	GeGLU	
Llama 3 (70B)	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Reka Flash	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Command R+	2024	LayerNorm	Parallel	■	RoPE	SwiGLU	
DLMo	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Qwen (72B)	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
DeepSeek (67B)	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Yi (34B)	2024	RMSNorm	Serial	■	RoPE	SwiGLU	
Mistral of Experts	2024			☐			
Command A	2025	LayerNorm	Parallel	■	Hybrid (RoPE+NoPE)	SwiGLU	
Gemma 3	2025	RMSNorm	Serial	☐	RoPE	GeGLU	Pre+post norm, QK-norm
SmolLM3 (1.7B)	2025	RMSNorm	Serial	■	NoPE	SwiGLU	

We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

WHAT ARE WE GOING TO COVER?

- Common architecture variations
  - Activations, FFN
  - Attention variants
  - Position embeddings
- Hyperparameters that (do or don't) matter
  - What is ff_dim? Do multi_head dims always sum to model_dim?
  - How many vocab elements?
- Stability tricks

Architecture Variations: Normalization

ARCHITECTURE VARIATIONS..

- Let's think about the core architecture piece
  - Trends toward 'LLaMA- like' architectures

Name	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)	2019	RMSNorm	Serial	☑	Relative	ReLU
GPT2	2019	LayerNorm	Serial	☑	Sine	GeLU
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	☑	Relative	GeGLU
mT5	2020	RMSNorm	Serial	☑	Relative	GeGLU
GPT3 (175B)	2020	LayerNorm	Serial	☑	Sine	GeLU
GPTJ	2021	LayerNorm	Parallel	☑	RoPE	GeLU
LaMDA	2021			☑	Relative	GeGLU
Gopher (280B)	2021	RMSNorm	Serial	☑	Relative	ReLU
GPT-NeoX	2022	LayerNorm	Parallel	☑	RoPE	GeLU
BLOOM (175B)	2022	LayerNorm	Serial	☑	ALiBi	GeLU
OPT (175B)	2022	LayerNorm	Serial	☑	Absolute	ReLU
PaLM (540B)	2022	RMSNorm	Parallel	☑	RoPE	SwiGLU
Chinchilla	2022	RMSNorm	Serial	☑	Relative	ReLU
Mistral (7B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA2 (70B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA (65B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
Qwen (14B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
DeepSeek (67B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Yi (34B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU

High level view:

- Low consensus (except pre-norm)
- Trends toward 'LLaMA-like' architectures

PRE-VS-POST NORM

The one thing *everyone* agrees on (in 2024)

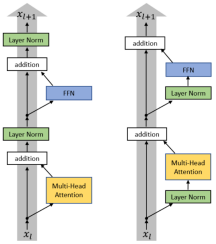


Figure from Xiong 2020

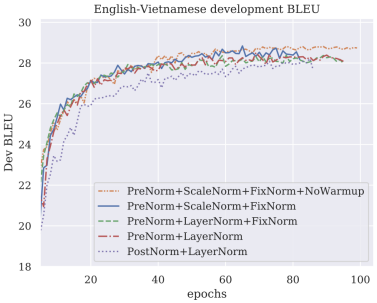
Post-LN Transformer	Pre-LN Transformer
$x_{l,i}^{post,1} = \text{MultiHeadAtt}(x_{l,i}^{post}, [x_{l,1}^{post}, \cdots, x_{l,n}^{post}])$	$x_{l,i}^{pre,1} = \text{LayerNorm}(x_{l,i}^{pre})$
$x_{l,i}^{post,2} = x_{l,i}^{post} + x_{l,i}^{post,1}$	$x_{l,i}^{pre,2} = \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \cdots, x_{l,n}^{pre,1}])$
$x_{l,i}^{post,3} = \text{LayerNorm}(x_{l,i}^{post,2})$	$x_{l,i}^{pre,3} = x_{l,i}^{pre} + x_{l,i}^{pre,2}$
$x_{l,i}^{post,4} = \text{ReLU}(x_{l,i}^{post,3} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$	$x_{l,i}^{pre,4} = \text{LayerNorm}(x_{l,i}^{pre,3})$
$x_{l,i}^{post,5} = x_{l,i}^{post,3} + x_{l,i}^{post,4}$	$x_{l,i}^{pre,5} = \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$
$x_{l+1,i}^{post} = \text{LayerNorm}(x_{l,i}^{post,5})$	$x_{l+1,i}^{pre} = x_{l,i}^{pre,5} + x_{l,i}^{pre,3}$
	Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$

Set up LayerNorm so that it doesn't affect the main residual signal path (on the left)

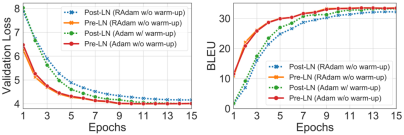
Almost all modern LMs use pre-norm (but BERT was post-norm)

(One somewhat funny exception – OPT350M. I don't know why this is post-norm)

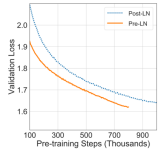
PRE-VS-POST-NORM, THE DATA



Salazar and Ngyuen 2019



(a) Validation Loss (IWSLT) (b) BLEU (IWSLT)

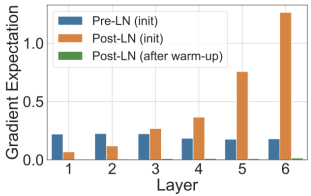


(a) Validation Loss on BERT

Figure from Xiong 2020

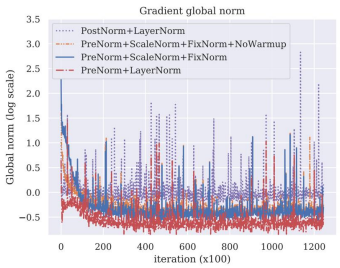
PRE-VS-POST NORM, EXPLANATIONS?

Gradient attenuation [Xiong 2020]



(a) W^1 in the FFN sub-layers

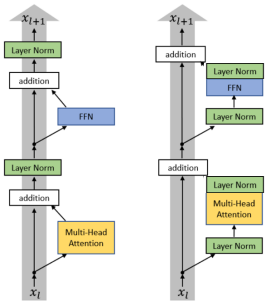
Gradient spikes [Salazar and Ngyuen]



Original stated advantage – removing warmup.
Today – stability and larger LRs for large networks

NEW THINGS – 'DOUBLE' NORM.

- If putting LayerNorms in residual streams is bad.. Why not post-norm outside the stream?
- Recent models: Grok, Gemma 2. Olmo 2 only does non-residual post norm



LAYERNORM VS RMSNORM

		Notable models
Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}	$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$	GPT3/2/1, OPT, GPT-J, BLOOM
Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term	$y = \frac{x}{\sqrt{\|x\|_2^2 + \varepsilon}} * \gamma$	LLaMA-family, PaLM, Chinchilla, T5

COMPUTE VS. MEMORY

- A GPU can do arithmetic much faster than it can move data between its memory and its compute units.
- The number of arithmetic operations per byte moved is an operation's **arithmetic intensity**. With low arithmetic intensity, an operation is limited by **memory traffic**, not by the number of floating-point operations (FLOPs).
- Matrix multiplications do many operations per byte loaded; normalizations and element-wise operations do few.
- So an operation can account for a tiny share of the FLOPs and still a noticeable share of the runtime — the next slides use exactly this argument.
- Hardware details follow in session 12.

WHY RMSNORM?

- Modern explanation – it's faster (and just as good).
  - Fewer operations (no mean calculation)
  - Fewer parameters (no bias term to store)
- Does this explanation make sense?
  - Matrix multiplies are the vast majority of FLOPs (and memory)

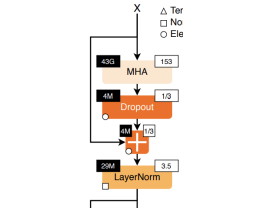
$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Operator class	% flop
△ Tensor contraction	99.80
□ Stat. normalization	0.17
○ Element-wise	0.03

[Ivanov et al 2023]

WHY RMSNORM (2)

- Important lesson: FLOPS are not runtime!
  - RMSNorm can still matter due to the importance of data movement



Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

Left top ("43G") is FLOPS Right top ("153") is the FLOP-to-memory ratio; [Ivanov et al 2023]

RMSNORM - VALIDATION

- **RMSNorm** runtime (and surprisingly, perf) gains have been seen in papers

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ	WMT EnDe
Vanilla Transformer	$223M$	$11.1T$	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02	26.62
RMS Norm	$223M$	$11.1T$	3.68	2.167 ± 0.008	**1.821**	**75.45**	**17.94**	**24.07**	**27.14**
Rezero	$223M$	$11.1T$	3.51	2.262 ± 0.003	1.939	61.69	15.64	20.90	26.37
Rezero + LayerNorm	$223M$	$11.1T$	3.26	2.223 ± 0.006	1.858	70.42	17.58	23.02	26.29
Rezero + RMS Norm	$223M$	$11.1T$	3.34	2.221 ± 0.009	1.875	70.33	17.32	23.02	26.19
Fixup	$223M$	$11.1T$	2.95	2.382 ± 0.012	2.067	58.56	14.42	23.02	26.31

Narang et al 2020

MORE GENERALLY: DROPPING BIAS TERMS

Most modern transformers don't have bias terms.

Original Transformer:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$$

Most implementations (if they're not gated):

$$FFN(x) = \sigma(xW_1)\, W_2$$

Reasons: memory (similar to RMSnorm) and optimization stability

LAYERNORM: RECAP

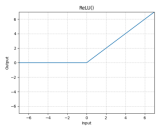
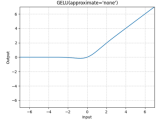
- Basically everyone does pre-norm.
- Intuition – keep the good parts of residual connections
- Observations – nicer gradient propagation, fewer spike
- Some people add a second norm outside the residual stream (NOT post-norm)
- Most people do RMSnorm
- In practice, works as well as LayerNorm
- But, has fewer parameters to move around, which saves on wallclock time
- People more generally drop bias terms since the compute/param tradeoffs are not great.

Architecture Variations: Activations and Layers

ACTIVATIONS

- A whole zoo of activations ..
- ReLU, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, SeLU, SwiGLU, LiGLU
  - What are these things? What do people use? Does it matter?

A FEW OF THE COMMON ACTIVATIONS

			Notable models
ReLU	$FF(x) = \max(0, xW_1)\, W_2$		Original transformer, T5, Gopher, Chinchilla, OPT
GeLU	$FF(x) = \text{GELU}(xW_1)\, W_2$ $\text{GELU}(x) := x\,\Phi(x)$		GPT1/2/3, GPTJ, GPT-Neox, BLOOM
SwiGLU / GeGLU (next slide..)			Llama, PaLM, T5 v1.1, *most models post 2023*

GATED ACTIVATIONS (*GLU)

GLUs modify the 'first part' of a FF layer

$$FF(x) = \max(0, xW_1)\, W_2$$

Instead of a linear + ReLU, augment the above with an (entrywise) linear term

$$\max(0, xW_1) \to \max(0, xW_1) \otimes (xV)$$

This gives the gated variant (ReGLU) – note that we have an extra parameter (V)

$$\mathrm{FF}_{\mathrm{ReGLU}}(x) = (\max(0, xW_1) \otimes xV)\, W_2$$

GATED VARIANTS OF STANDARD FF LAYERS

	Notable models
GeGLU $\text{FFN}_{\text{GEGLU}}(x, W, V, W_2) = (\text{GELU}(xW) \otimes xV)\, W_2$	T5 v1.1, mT5, LaMDA, Phi3, Gemma 2, Gemma 3
SwiGLU (swish is $x * \text{sigmoid}(x)$) $\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)\, W_2$	LLaMa 1/2/3, PaLM, Mistral, OlMo, *most models post 2023*

Note: Gated models use smaller dimensions for the d_{ff} by 2/3

DO GATED LINEAR UNITS WORK?

Yes, fairly consistently so.

	Score Average	CoLA MCC	SST-2 Acc
FFN_{ReLU}	83.80	51.32	94.04
FFN_{GELU}	83.86	53.48	94.04
$\text{FFN}_{\text{Swish}}$	83.60	49.79	93.69
FFN_{GLU}	84.20	49.16	94.27
$\text{FFN}_{\text{GEGLU}}$	84.12	53.65	93.92
$\text{FFN}_{\text{Bilinear}}$	83.79	51.02	**94.38**
$\text{FFN}_{\text{SwiGLU}}$	84.36	51.59	93.92
$\text{FFN}_{\text{ReGLU}}$	**84.67**	**56.16**	**94.38**
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Shazeer 2020

DO GATED LINEAR UNITS WORK (2)?

- Yes, with other works corroborating Shazeer 2020
  - Narang et al 2020

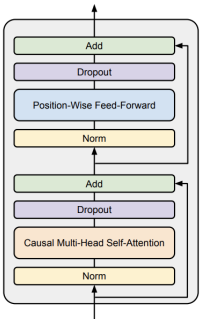
Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ
Vanilla Transformer	$223M$	$11.1T$	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02
GeLU	$223M$	$11.1T$	3.58	2.179 ± 0.003	1.838	**75.79**	**17.86**	**25.13**
Swish	$223M$	$11.1T$	3.62	2.186 ± 0.003	1.847	**73.77**	17.74	**24.34**
ELU	$223M$	$11.1T$	3.56	2.270 ± 0.007	1.932	67.83	16.73	23.02
GLU	$223M$	$11.1T$	3.59	2.174 ± 0.003	**1.814**	**74.20**	**17.42**	24.34
GeGLU	$223M$	$11.1T$	3.55	2.130 ± 0.006	**1.792**	**75.96**	**18.27**	**24.87**
ReGLU	$223M$	$11.1T$	3.57	2.145 ± 0.004	**1.803**	**76.17**	**18.36**	**24.87**
SeLU	$223M$	$11.1T$	3.55	2.315 ± 0.004	1.948	68.76	16.76	22.75
SwiGLU	$223M$	$11.1T$	3.53	2.127 ± 0.003	**1.789**	**76.00**	**18.20**	**24.34**
LiGLU	$223M$	$11.1T$	3.59	2.149 ± 0.005	**1.798**	**75.34**	**17.97**	**24.34**
Sigmoid	$223M$	$11.1T$	3.63	2.291 ± 0.019	1.867	**74.31**	17.51	23.02
Softplus	$223M$	$11.1T$	3.47	2.207 ± 0.011	1.850	**72.45**	17.65	**24.34**

GATING, ACTIVATIONS

- Many variations (ReLU, GeLU, *GLU) across models.
- *GLU isn't necessary for a good model (see GPT3), but it's probably helpful Otuer, recent outlier models.. Nemotron 340B (Squared ReLU), Falcon 2 11b (ReLU)
- But evidence points towards somewhat consistent gains from Swi/GeGLU

SERIAL VS PARALLEL LAYERS

- Normal transformer blocks are serial – they compute attention, then the MLP
  - Could we parallelize the transformer block?



PARALLEL LAYERS

- A few models (GPTJ, PaLM, GPT-NeoX) do parallel layers. Originally in GPT-J
- If implemented right, LayerNorm can be shared, and matrix multiplies can be fused
- Recent Models: Cohere Command A, Falcon 2 11B, Command R+

Parallel Layers – We use a "parallel" formulation in each Transformer block (Wang & Komatsuzaki, 2021), rather than the standard "serialized" formulation. Specifically, the standard formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

Whereas the parallel formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused. Ablation experiments showed a small quality degradation at 8B scale but no quality degradation at 62B scale, so we extrapolated that the effect of parallel layers should be quality neutral at the 540B scale.

SUMMARY: ARCHITECTURES

Pre-vs-post norm:

- Everyone does pre-norm (except OPT350M), likely with good reason.

Layer vs RMSnorm:

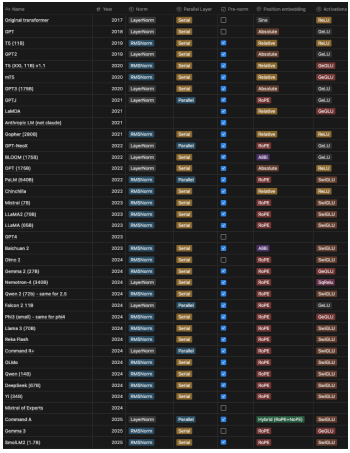
- RMSnorm has clear compute wins, sometimes even performance

Gating:

- GLUs seem generally better, though differences are small

Serial vs parallel layers:

- No extremely serious ablations, but has a compute win.



Name	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)	2019	RMSNorm	Serial	☑	Relative	ReLU
GPT2	2019	LayerNorm	Serial	☑	Absolute	GeLU
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	☑	Relative	GeGLU
mT5	2020	RMSNorm	Serial	☑	Relative	GeGLU
GPT3 (175B)	2020	LayerNorm	Serial	☑	Absolute	GeLU
GPTJ	2021	LayerNorm	Parallel	☑	RoPE	GeLU
LaMDA	2021			☑	Relative	GeGLU
Anthropic LM (not clear)	2021			☑		
Gopher (280B)	2021	RMSNorm	Serial	☑	Relative	ReLU
GPT-Neox	2022	LayerNorm	Parallel	☑	RoPE	GeLU
BLOOM (175B)	2022	LayerNorm	Serial	☑	ALiBi	GeLU
OPT (175B)	2022	LayerNorm	Serial	☑	Absolute	ReLU
PaLM (540B)	2022	RMSNorm	Parallel	☑	RoPE	SwiGLU
Chinchilla	2022	RMSNorm	Serial	☑	Relative	ReLU
Mistral (7B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLAMA2 (70B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLAMA (65B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
GPT4	2023			☐		
Reka/Yuan 2	2023	RMSNorm	Serial	☑	ALiBi	SwiGLU
Qwen 2	2024	RMSNorm	Serial	☐	RoPE	SwiGLU
Gemma 2 (27B)	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Nemotron-4 (340B)	2024	LayerNorm	Serial	☑	RoPE	Sqrelu
Qwen 2 (72b) - same for 2.5	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Falcon 2 11B	2024	LayerNorm	Parallel	☑	RoPE	GeLU
PALI (small) - same for pilot	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Llama 3 (174b)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Reka Flash	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Command R+	2024	LayerNorm	Parallel	☑	RoPE	SwiGLU
DLMo	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Qwen (14B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
DeepSeek (67B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Yi (34B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Mistral of Experts	2024			☐		
Command A	2025	LayerNorm	Parallel	☑	Hybrid (RoPE+NoPE)	SwiGLU
Gemma 3	2025	RMSNorm	Serial	☐	RoPE	GeGLU
MiniLM2 (1.7B)	2025	RMSNorm	Serial	☑	RoPE	SwiGLU

Position Embeddings: RoPE

MANY VARIATIONS IN POSITION EMBEDDINGS

		Notable models
Sine embeddings: add sines and cosines that enable localization	$Embed(x,i) = v_x + PE_{pos}$ $PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$ $PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$	Original transformer
Absolute embeddings: add a position vector to the embedding	$Embed(x,i) = v_x + u_i$	GPT1/2/3, OPT
Relative embeddings: add a vector to the *attention computation*	$e_{ij} = \frac{x_i W^Q \left(x_j W^K + a_{ij}^K\right)^T}{\sqrt{d_z}}$	T5, Gopher, Chinchilla
Rope embeddings (next slides..)		GPTJ, PaLM, LLaMA, *Most 2024+ models*

ROPE: ROTARY POSITION EMBEDDINGS

High level thought process: a *relative* position embedding should be some $f(x, i)$ s.t.

$$\langle f(x,i), f(y,j) \rangle = g(x, y, i-j)$$

That is, the attention function *only* gets to depend on the relative position $(i-j)$. How do existing embeddings not fulfill this goal?

- **Sine:** Has various cross-terms that are not relative

$$\langle Embed(x,i), Embed(y,i) \rangle = \langle v_x, v_y \rangle + \langle PE_i, v_y \rangle \ldots$$

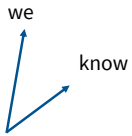
- **Absolute:** obviously not relative
- **Relative embeddings:**

$$e_{ij} = \frac{x_i W^Q \left(x_j W^K + a_{ij}^K\right)^T}{\sqrt{d_z}}$$ is not an inner product

ROPE: ROTARY POSITION EMBEDDINGS

How can we solve this problem?

- We want our embeddings to be **invariant to absolute position**
- We know that inner products are invariant to arbitrary rotation.

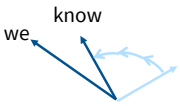


Position independent embedding



Embedding "we know that"

Rotate we by '0 positions'
know by '1 positions'

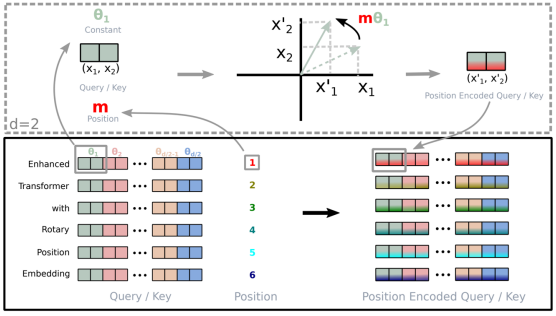


Embedding "of course we know"

Rotate we by '2 positions'
Rotate know by '3 positions'

ROPE: ROTARY POSITION EMBEDDINGS

- There are many rotations, which one do you pick?
  - Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)



[Su et al 2021]

THE ACTUAL ROPE MATH

- Multiply with sines and cosines
- Difference with sine embeddings – not additive, no cross terms

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m} \boldsymbol{W}_{\{q,k\}} \boldsymbol{x}_m \tag{14}$$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{15}$$

IMPLEMENTATION AND CODE FOR ROPE

▷ Usual attention stuff

```
query_states = self.q_proj(hidden_states)
key_states = self.k_proj(hidden_states)
value_states = self.v_proj(hidden_states)

# Flash attention requires the input to have the shape
# batch_size x seq_length x head_dim x hidden_dim
# therefore we just need to keep the original shape
query_states = query_states.view(bsz, q_len, self.num_heads, self.head_dim).transpose(1, 2)
key_states = key_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)
value_states = value_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)
```

▷ Get the RoPE matrix cos/sin

```
cos, sin = self.rotary_emb(value_states, position_ids)
```

▷ Multiply query/key inputs

```
query_states, key_states = apply_rotary_pos_emb(query_states, key_states, cos, sin)
```

. . . Same stuff as the usual multi-head self attention below

Note: embedding at *each attention operation* to enforce position invariance

Hyperparameters

HYPERPARAMETERS

- Transformer hyperparameter questions you might have had
- How much bigger should the feedforward size be compared to hidden size?
- How many heads, and should num_heads always divide hidden size?
- What should my vocab size be?
- And other model setting questions
  - Do people even regularize these huge LMs?
  - How do people scale these models - very deep or very wide?

SURPRISING (?) CONSENSUS HYPERPARAMETER 1

Feedforward – model dimension ratio.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\boldsymbol{d_{ff} = 4\, d_{model}}$$

This is *almost always* true. There's just a few exceptions.

EXCEPTION #1 – GLU VARIANTS

Remember that GLU variants scale down by 2/3rd. This means most GLU variants have $d_{ff} = \frac{8}{3} d_{model}$. This is mostly what happens. Some notable such examples.

Model	d_{ff} / d_{model}
PaLM	4
Mistral 7B	3.5
LLaMA-2 70B	3.5
LLaMA 70B	2.68
Qwen 14B	2.67
DeepSeek 67B	2.68
Yi 34B	2.85
T5 v1.1	2.5

Models are roughly in this range, though PaLM, LLaMA2 and Mistral are slightly larger

EXCEPTION #2 – T5

As we have (and will) see, most LMs are have boring, conservative hyperparameters. One exception is T5 [Raffel et al 2020] which has some *very bold* settings.

In particular, for the 11B model, they set

$$d_{ff} = 65{,}536 \qquad d_{model} = 1024$$

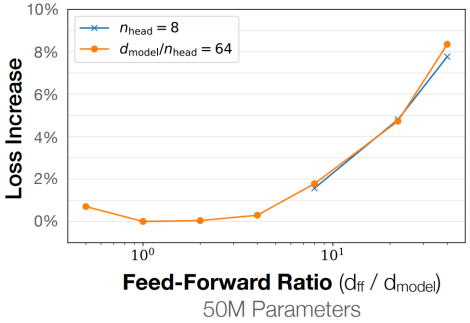
For an astounding 64-times multiplier.

> for "11B" we use $d_{\text{ff}} = 65{,}536$ with 128-headed attention producing a model with about 11 billion parameters. We chose to scale up d_{ff} specifically because modern accelerators (such as the TPUs we train our models on) are most efficient for large dense matrix multiplications like those in the Transformer's feed-forward networks.

Other, recent exceptions – Gemma 2 (8x), SmolLM/Gemma 3 (4x, GLU)

WHY THIS RANGE OF MULTIPLIERS?

- Empirically, there's a basin between 1-10 where this hyperparameter is near-optimal



[Kaplan+ 2020]

WHAT CAN WE LEARN FROM THE MODEL-DIM HYPERPARAM?

- The 'default' choices of $d_{ff} = 4d_{model}$ and $d_{ff} = 2.66d_{model}$ have worked well for nearly all modern LLMs.
- But T5 does show that even radical choices of $d_{ff} = 64d_{model}$ can work. This hyperparameter choice isn't written in stone.
- That said, T5 has a follow-up model (T5 v1.1) that is 'improved' and uses a much more standard 2.5 multiplier on GeGLU, so the 64-times multiplier is likely suboptimal.

SURPRISING (?) CONSENSUS HYPERPARAMETER 2

- Head-dim*num-heads to model-dim ratio.
- This doesn't have to be true: we can have head-dimensions > model-dim / num-heads.
  - But most models do follow this guideline

Multi-head self-attention is computationally efficient

- Even though we compute h many attention heads, it's not really more costly.
  - We compute $XQ \in \mathbb{R}^{n \times d}$, and then reshape to $\mathbb{R}^{n \times h \times d/h}$. (Likewise for XK, XV.)
  - Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
  - Almost everything else is identical, and the **matrices are the same sizes.**

HOW MANY HEADS, WHATS THE MODEL DIM?

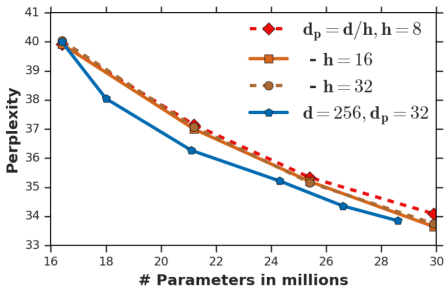
- Some examples of this hyperparameter

	Num heads	Head dim	Model dim	Ratio
GPT3	96	128	12288	1
T5	128	128	1024	16
T5 v1.1	64	64	4096	1
LaMDA	128	128	8192	2
PaLM	48	258	18432	1.48
LLaMA2	64	128	8192	1

Most models have ratios around 1 – notable exceptions by some google models.

EVIDENCE FOR 1-1 RATIO?

- There have been papers written against the 1-1 ratio [Bhojanapalli et al 2020]
- But we don't seem to be seeing significant 'low rank bottlenecks' in practice..



ASPECT RATIOS

- Should my model be deep or wide? *How* deep and how wide?
- Most models are surprisingly consistent on this one too!

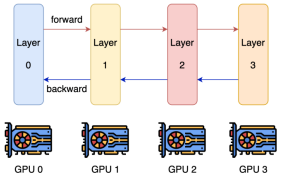
	Model	d_{model} / n_{layer}
	BLOOM	205
	T5 v1.1	171
	PaLM (540B)	156
Sweet spot?	GPT3/OPT/Mistral/Qwen	128
	LLaMA / LLaMA2 / Chinchila	102
	T5 (11B)	43
	GPT2	33

CONSIDERATIONS ABOUT ASPECT RATIO

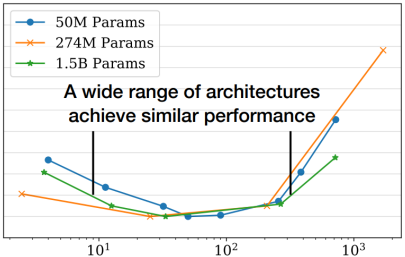
- Extremely deep models are harder to parallelize and have higher latency

> **The Limits of Depth vs Width** We note an obvious limitation with our advice. Scaling depth has an obvious limiter, i.e., they are non-parallelizable across different machines or devices and every computation has to always wait for the previous layer. This is unlike width, which can be easily parallelizable over thousands or hundreds of thousands of devices. Within the limitation of scaling

[Tay et al 2021]

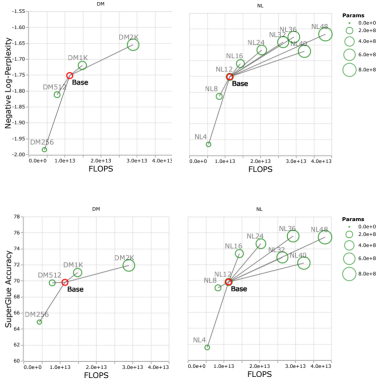


EVIDENCE ON ASPECT RATIO SCALING



Aspect Ratio (d_{model} / n_{layer})

[Kaplan et al 2020]



[Tay et al 2021]

WHAT ARE TYPICAL VOCABULARY SIZES?

Monolingual models – 30-50k vocab

Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000

Multilingual / production systems 100-250k

Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Command A	255000
DeepSeek	100000
Qwen 15B	152064
Yi	64000

Monolingual vocabs don't need to be huge, but multilingual ones do

DROPOUT AND OTHER REGULARIZATION

- Do we need regularization during pretraining?
- Arguments against:
  - There is a lot of data (trillions of tokens), more than parameters.
  - SGD only does a single pass on a corpus (hard to memorize)
- This is all quite reasonable.. but what do people do in practice?

DROPOUT AND WEIGHT DECAY IN PRACTICE

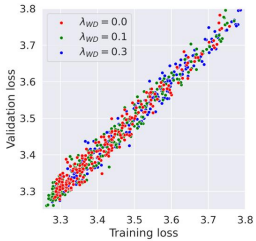
Model	Dropout*	Weight decay
Original transformer	0.1	0
GPT2	0.1	0.1
T5	0.1	0
GPT3	0.1	0.1
T5 v1.1	0	0
PaLM	0	(variable)
OPT	0.1	0.1
LLaMA	0	0.1
Qwen 14B	0.1	0.1

- Many older models used dropout during pretraining
- Newer models (except Qwen) rely only on weight decay

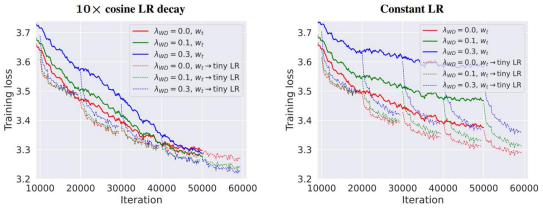
* Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout. This may not be true of closed models.

WHY WEIGHT DECAY LLMS?

[Andriushchenko et al 2023] has interesting observations about LLM weight decay



It's not to control overfitting



Weight decay interacts with learning rates (cosine schedule)

SUMMARY: HYPERPARAMETERS

Feedforward

- Factor-of-4 rule of thumb (8/3 for GLUs) is standard (with some evidence)

Head dim

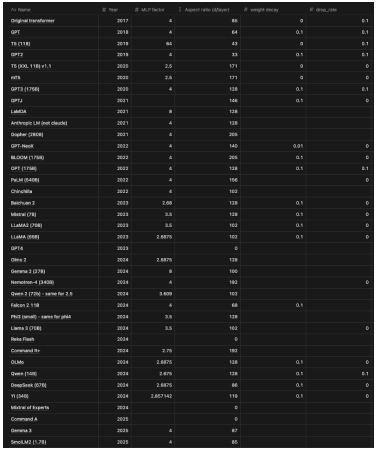
- Head dim*Num head = D model is standard – but low to no validation

Aspect ratio

- Wide range of 'good' values (100-200). Systems concerns dictate the value

Regularization

- You still 'regularize' LMs but its effects are primarily on optimization dynamics

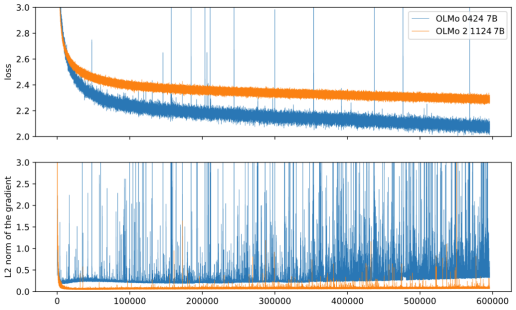


Arc Name	Year	MLP factor	Aspect ratio (d/layer)	weight decay	drop_rate
Original transformer	2017	4	86	0	0.1
GPT	2018	4	64	0.1	0.1
T5 (11B)	2019	64	43	0	0.1
GPT2	2019	4	33	0.1	0.1
T5 (XXL 11B) v1.1	2020	2.5	171	0	0
mT5	2020	2.5	171	0	0
GPT3 (175B)	2020	4	128	0.1	0.1
GPTJ	2021		146	0.1	0
LaMDA	2021	8	128		
Anthropic LM (not disclo)	2021	4	128		
Gopher (280B)	2021	4	205		
GPT-Neox	2022	4	140	0.01	0
BLOOM (176B)	2022	4	205	0.1	0
OPT (175B)	2022	4	128	0.1	0.1
PaLM (540B)	2022	4	156		0
Chinchilla	2022	4	102		
Baichuan 2	2023	2.66	128	0.1	0
Mistral (7B)	2023	3.5	128	0.1	0
LLaMA2 (70B)	2023	3.5	102	0.1	0
LLaMA (65B)	2023	2.6875	102	0.1	0
GPT4	2023		0		
Olmo 2	2024	2.6875	128		
Gemma 2 (27B)	2024	8	100		
Nemotron-4 (340B)	2024	4	192		0
Qwen 2 (72b) - same for 2.5	2024	3.809	102		
Falcon 2 11B	2024	4	68	0.1	
Phi3 (small) - same for phi4	2024	3.5	128		
Llama 3 (70B)	2024	3.5	102		0
Reka Flash	2024		0		
Command R+	2024	2.75	192		
OLMo	2024	2.6875	128	0.1	0
Qwen (110)	2024	2.875	128	0.1	0.1
DeepSeek (67B)	2024	2.6875	86	0.1	0
Yi (34B)	2024	2.857142	119	0.1	0
Mistral of Experts	2024		0		
Command A	2025		0		
Gemma 3	2025	4	87		
SmolLM2 (1.7B)	2025	4	85		

Stability Tricks

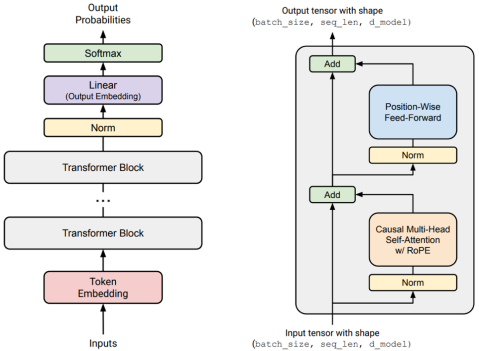
STABILITY TRICKS

- Recently, lots of attention on stable training
  - Don't train models that look like the blue curve!



WHERE DO THE ISSUES ARISE? BEWARE OF SOFTMAXES!

Softmaxes – can be ill-behaved due to exponentials / division by zero



OUTPUT SOFTMAX STABILITY – THE 'Z-LOSS'

Recall the softmax calculation

$$\begin{aligned} \log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \Sigma_{r'=1}^{|V|} e^{U_{r'}(x)} \end{aligned}$$

$$\begin{aligned} L &= \sum_i \left[\log(P(x_i)) - \alpha(\log(Z(x_i)) - 0)^2\right] \\ &= \sum_i \left[\log(P(x_i)) - \alpha \log^2(Z(x_i))\right] \end{aligned}$$

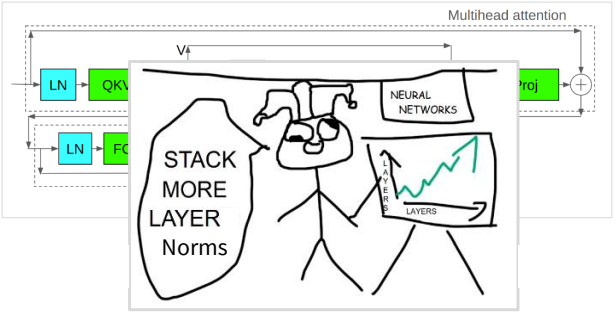
[From Devlin 2014]

This is useful for stability! PaLM pioneered this 'z loss' trick.

> We additionally use an auxiliary loss of $z_loss = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2 (2025)

ATTENTION SOFTMAX STABILITY – THE 'QK NORM'



The query and keys are Layer (RMS) normed before going into the softmax operation.

Other examples: DCLM, OLMo2, Gemma 2

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]

LOGIT SOFT-CAPPING.

- Soft-capping the logits to some maximum value via Tanh
- Prevents logits from blowing up, but also might have perf issues?

Logit soft-capping. We cap logits (Bello et al., 2016) in each attention layer and the final layer such that the value of the logits stays between −soft_cap and +soft_cap. More specifically, we cap the logits with the following function:

$$\text{logits} \leftarrow \text{soft_cap} * \tanh(\text{logits}/\text{soft_cap}).$$

We set the soft_cap parameter to 50.0 for the self-attention layers and to 30.0 for the final layer.

Table 4: Models perplexity with confidence interval ±0.1 at 95% level.

bf16 baseline	*soft_cap*	*QKV_norm*	*QK_norm_cap*	*QK_norm*	*QK_FC_norm*
11.19	11.24	10.85	11.00	10.84	10.87

Attention Heads: MQA, GQA, Sliding Windows

ATTENTION HEADS

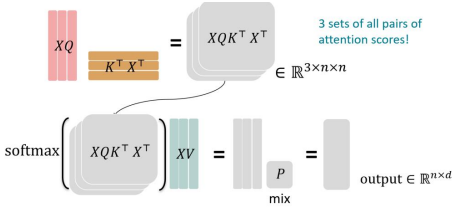
- Most models don't touch the attention heads much at all with a few minor exceptions..
- GQA / MQA : Saving inference costs by reducing the number of heads
- Sparse or sliding window attention (GPT4/Mistral): restricting the attention pattern
  - to reduce compute cost
- Exotic SSM stuff (Jamba, Falcon 3, etc): not covered (sorry!)

THE KV CACHE

- Autoregressive generation (session 8, decoding) produces one token at a time; each new token attends to all previous tokens.
- The keys and values of the previous tokens do not change, so they are stored instead of recomputed: the **KV cache**.
- Its size grows with sequence length $\times$ number of layers $\times$ number of heads $\times$ head dimension (per sequence in the batch).
- At inference time, reading this cache from memory — not the arithmetic — is often the bottleneck. The following attention variants reduce it.

GQA/MQA – REDUCING ATTENTION HEAD COST

Let's think about the compute involved for attention



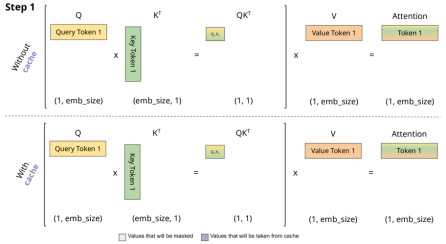
Total arithmetic operations (bnd^2)**, total memory accesses**

$$\underset{(bnd}{\text{X}} + \underset{bhn^2}{\text{softmax}} + \underset{d^2}{\text{projection}} \quad)$$

Arithmetic intensity is high $O\left(\left(\frac{1}{k}+\frac{1}{bn}\right)^{-1}\right)$ – we can keep our GPUs running

GQA/MQA – REDUCING ATTENTION HEAD COST

- What about the incremental case when we generate text?
- Key difference: can't parallelize the generation process – needs to be step by step
- In this case – we need to incrementaly re-compute/update attention via the 'KV cache'



[Animation from https://medium.com/@joaolages/kv-caching-explained-276520203249]

GQA/MQA – REDUCING ATTENTION HEAD COST

What's the incremental arithmetic intensity?

Total arithmetic operations (bnd^2), **total memory accesses**

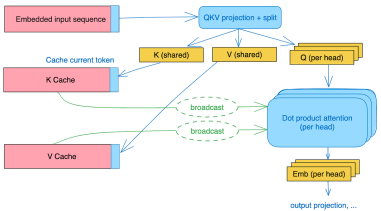
$$(bn^2d + \overbrace{nd^2}^{\text{projection}})$$

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ – need large batches + short seq length (n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.

MQA – JUST HAVE FEWER KEY DIMENSIONS.

Key idea – have multiple queries, but just one dimension for keys and values



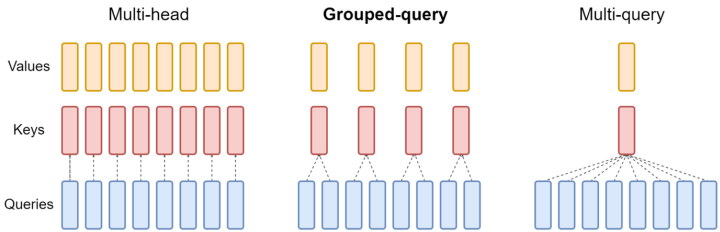
We have much fewer items to move in and out of memory (KV Cache)

Total memory access $(bnd + bn^2k + nd^2)$, **Arithmetic intensity** $O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$

[figure from https://blog.fireworks.ai/multi-query-attention-is-all-you-need-db072e758055]

RECENT EXTENSION – GQA

- Don't go all the way to one dimension of KV – have fewer dims
- Simple knob to control expressiveness (key-query ratio) and inference efficiency



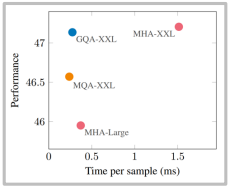
DOES MQA HURT? SOMETIMES..

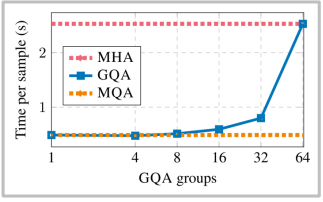
Small PPL hit w/ MQA [Shazeer 2019]

Table 3: Billion-Word LM Benchmark Results.

Attention	h	d_k, d_v	d_{ff}	dev-PPL
multi-head	8	128	8192	**29.9**
multi-query	8	128	9088	30.2
multi-head	1	128	9984	31.2
multi-head	2	64	9984	31.1
multi-head	4	32	9984	31.0
multi-head	8	16	9984	30.9

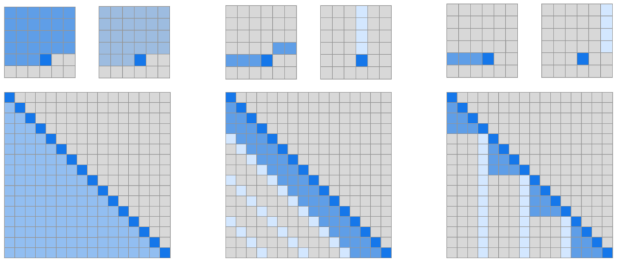
Low/no hit w/ GQA [Ainslie 2023]





SPARSE / SLIDING WINDOW ATTENTION

- Attending to the entire context can be expensive (quadratic).
- Build sparse / structured attention that trades off expressiveness vs runtime (GPT3)

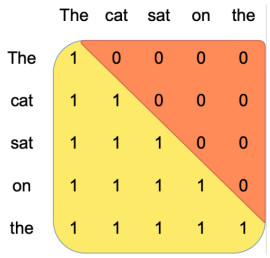


(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)

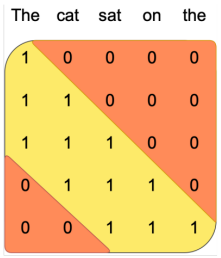
[Child et al 2019]

SLIDING WINDOW ATTENTION

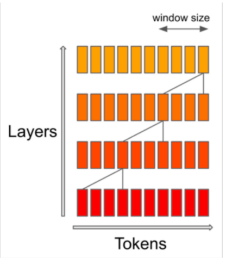
- Another variation on this idea – sliding window attention
- Just use the main part of the strided pattern – let depth extend effective context (Mistral)



Vanilla Attention



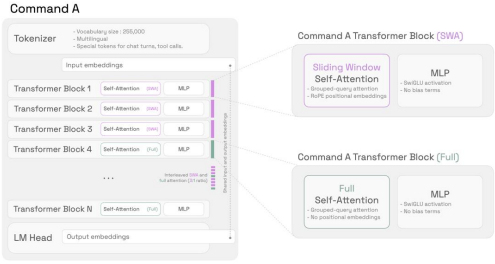
Sliding Window Attention



Effective Context Length

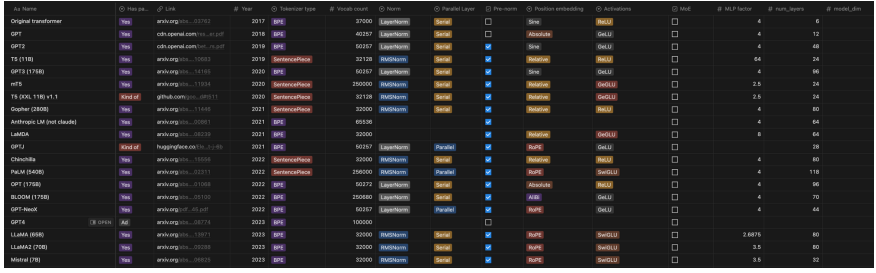
CURRENT STANDARD TRICK – INTERLEAVE 'FULL' AND 'LR' ATTENTION

- From Cohere Command A – Every 4th layer is a full attention
- Long-range info via NoPE, short-range info via RoPE + SWA.
- Other models – LLaMA 4, Gemma does SWA+Full RoPE.



RECAP, CONCLUSION, ETC.

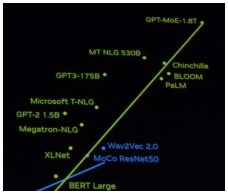
- Many aspects (arch, hparams) of transformers are in common across the big LMs
  - Major differences? Position embeddings, activations, tokenization



Name	Has pa...	Link	Year	Tokenizer type	Vocab count	Norm	Parallel Layer	Pre-norm	Position embedding	Activations	MoE	MLP factor	num_layers	model_dim
Original transformer	Yes	arxiv.org/abs...03762	2017	BPE	37000	LayerNorm	Serial	☐	Sine	ReLU	☐	4	6	
GPT	Yes	cdn.openai.com/res...er.pdf	2018	BPE	40257	LayerNorm	Serial	☐	Absolute	GeLU	☐	4	12	
GPT2	Yes	cdn.openai.com/bet...rs.pdf	2019	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	48	
T5 (11B)	Yes	arxiv.org/abs...10683	2019	SentencePiece	32128	RMSNorm	Serial	☑	Relative	ReLU	☐	64	24	
GPT3 (175B)	Yes	arxiv.org/abs...14165	2020	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	96	
mT5	Yes	arxiv.org/abs...11934	2020	SentencePiece	250000	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24	
T5 (XXL 11B) v1.1	Kind of	github.com/goo...dmt511	2020	SentencePiece	32128	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24	
Gopher (280B)	Yes	arxiv.org/abs...11446	2021	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80	
Anthropic LM (not claude)	Yes	arxiv.org/abs...00861	2021	BPE	65536			☑			☐	4	64	
LaMDA	Yes	arxiv.org/abs...08239	2021	BPE	32000			☑	Relative	GeGLU	☐	8	64	
GPTJ	Kind of	huggingface.co/Eleu...t-j-6b	2021	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐		28	
Chinchilla	Yes	arxiv.org/abs...15556	2022	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80	
PaLM (540B)	Yes	arxiv.org/abs...02311	2022	SentencePiece	256000	RMSNorm	Parallel	☑	RoPE	SwiGLU	☐	4	118	
OPT (175B)	Yes	arxiv.org/abs...01068	2022	BPE	50272	LayerNorm	Serial	☑	Absolute	ReLU	☐	4	96	
BLOOM (175B)	Yes	arxiv.org/abs...05100	2022	BPE	250680	LayerNorm	Serial	☑	ALiBi	GeLU	☐	4	70	
GPT-NeoX	Yes	arxiv.org/pdf...65.pdf	2022	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐	4	44	
GPT4 OPEN	Ad	arxiv.org/abs...08774	2023	BPE	100000			☐			☐			
LLaMA (65B)	Yes	arxiv.org/abs...13971	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	2.6875	80	
LLaMA2 (70B)	Yes	arxiv.org/abs...09288	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	80	
Mistral (7B)	Yes	arxiv.org/abs...06825	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	32	

Mixture of Experts

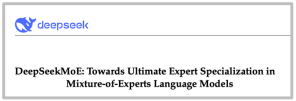
MIXTURE OF EXPERTS



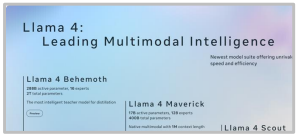
GPT4 (?)





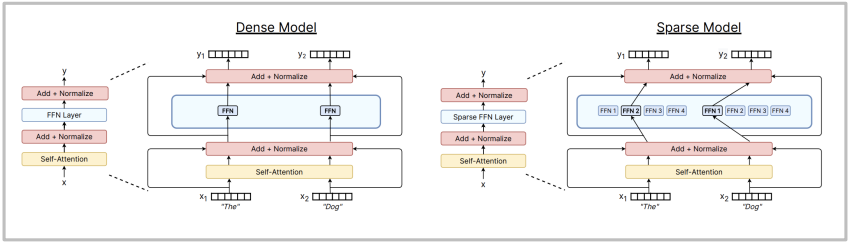








WHAT'S A MOE?

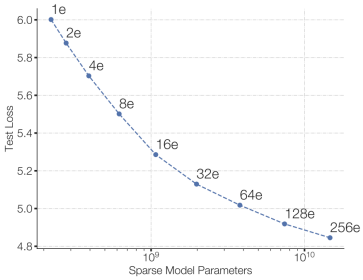


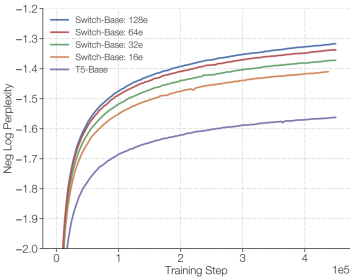
[Fedus et al 2022]

- Replace big feedforward with (many) big feedforward networks and a selector layer
  - You can increase the # experts without affecting FLOPs

WHY ARE MOES GETTING POPULAR?

- Same FLOP, more param does better

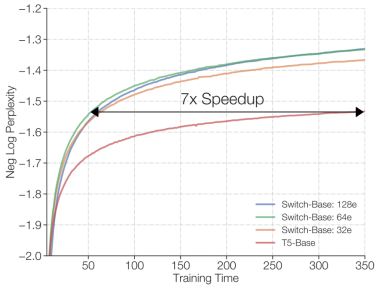




[Fedus et al 2022]

WHY ARE MOES GETTING POPULAR?

- Faster to train MoEs



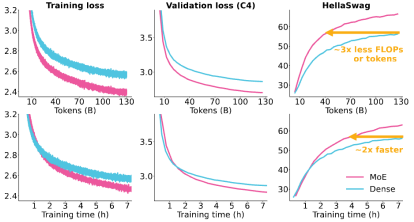
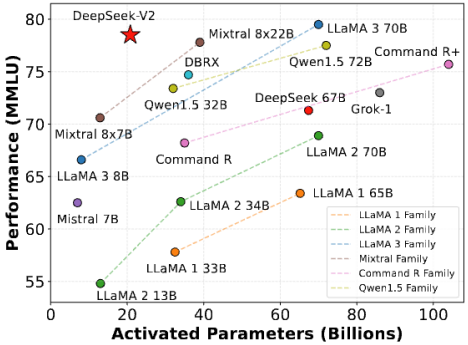


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

[OlMoE]

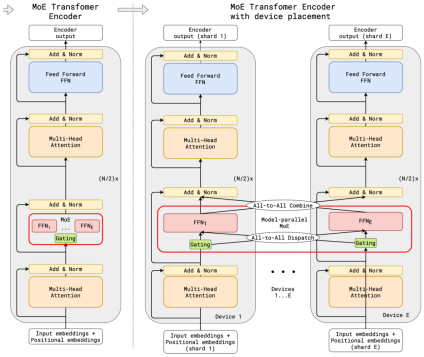
WHY ARE MOES GETTING POPULAR?

- Highly competitive vs dense equivalents



WHY ARE MOES GETTING POPULAR?

- Parallelizable to many devices



SOME MOE RESULTS – FROM THE WEST

- MoEs are most of the highest-performance open models, and are quite quick

Category / Benchmark	Llama 4 Maverick	Gemini 2.0 Flash	DeepSeek v3.1	GPT-4o
Inference Cost Cost per 1M input & output tokens (3:1 blended)	$0.19-$0.49[2]	**$0.17**	$0.48	$4.38
Image Reasoning MMMU	**73.4**	71.7	No multimodal support	69.1
MathVista	**73.7**	73.1		63.8
Image Understanding ChartQA	**90.0**	88.3		85.7
DocVQA (test)	**94.4**	—		92.8
Coding LiveCodeBench (10/01/2024-02/01/2025)	43.4	34.5	**45.8/49.2**[3]	32.3[3]
Reasoning & Knowledge MMLU Pro	80.5	77.6	**81.2**	—
GPQA Diamond	**69.8**	60.1	68.4	53.6

EARLIER MOE RESULTS FROM CHINESE GROUPS – QWEN

- Chinese LLM companies are also doing quite a bit of MoE work on the smaller end

Model	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	64.1	47.5	27.4	40.0	7.60
Gemma-7B	64.6	50.9	32.3	-	-
Qwen1.5-7B	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	62.5	61.5	34.2	40.8	7.17

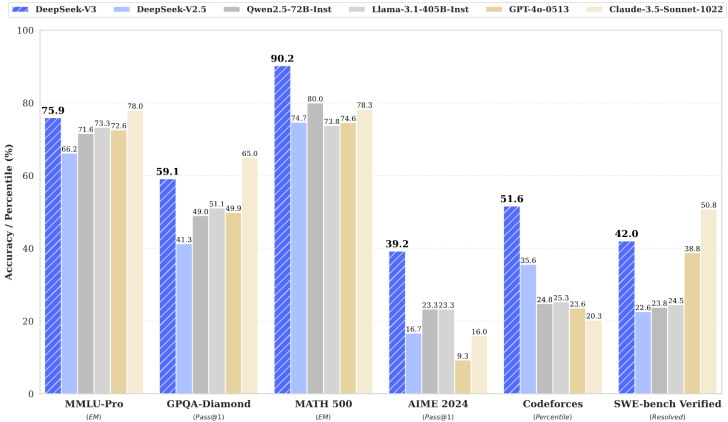
Model	#Parameters	#(Activated) Parameters
Mistral-7B	7.2	7.2
Qwen1.5-7B	7.7	7.7
Gemma-7B	8.5	7.8
DeepSeekMoE 16B	16.4	2.8
Qwen1.5-MoE-A2.7B	14.3	2.7

EARLIER MOE RESULTS FROM CHINESE GROUPS - DEEPSEEK

- There's also some good recent ablation work on MoEs showing they're generally good

Metric	# Shot	Dense	Hash Layer	Switch
# Total Params	N/A	0.2B	2.0B	2.0B
# Activated Params	N/A	0.2B	2.0B	0.2B
FLOPs per 2K Tokens	N/A	2.9T	2.9T	2.9T
# Training Tokens	N/A	100B	100B	100B
Pile (Loss)	N/A	2.060	1.932	1.881
HellaSwag (Acc.)	0-shot	38.8	46.2	49.1
PIQA (Acc.)	0-shot	66.8	68.4	70.5
ARC-easy (Acc.)	0-shot	41.0	45.3	45.9
ARC-challenge (Acc.)	0-shot	26.0	28.2	30.2
RACE-middle (Acc.)	5-shot	38.8	38.8	43.6
RACE-high (Acc.)	5-shot	29.0	30.0	30.9
HumanEval (Pass@1)	0-shot	0.0	1.2	2.4
MBPP (Pass@1)	3-shot	0.2	0.6	0.4
TriviaQA (EM)	5-shot	4.9	6.5	8.9
NaturalQuestions (EM)	5-shot	1.4	1.4	2.5

RECENT MOE RESULTS – DEEPSEEK V3



WHY HAVEN'T MOES BEEN MORE POPULAR?

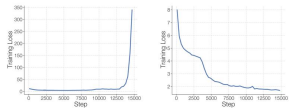
- Infrastructure is complex / advantages on multi node

> At a high level, sparsity is good when you have many accelerators (e.g. GPU/TPU) to host all the additional parameters that comes when using sparsity. Typically models are trained using data-parallelism where different machines will get different slices of the training/inference data. The machines used for operating on the different slices of data can now be used to host many more model parameters. Therefore, sparse models are good when training with data parallelism and/or have high throughput while serving: training/serving on many machines which can host all of the parameters.

[Fedus et al 2022]

- Training objectives are somewhat heuristic (and sometimes unstable)

Sparse models often suffer from training instabilities (Figure 1) worse than those observed in standard densely-activated Transformers.

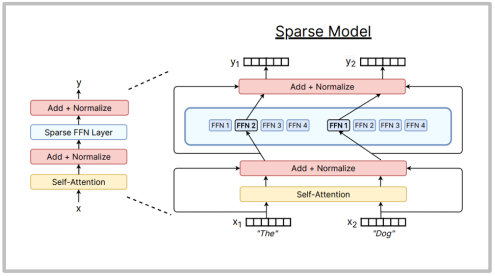


[Zoph et al 2022]

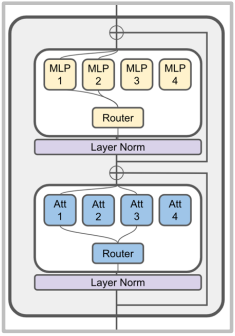
MoE Routing

WHAT MOES GENERALLY LOOK LIKE

Typical: replace MLP with MoE layer



Less common: MoE for attention heads



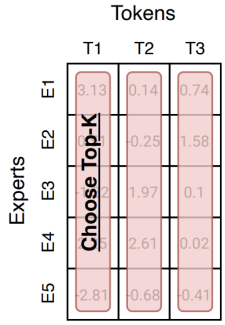
[ModuleFormer, JetMoE]

MOE – WHAT VARIES?

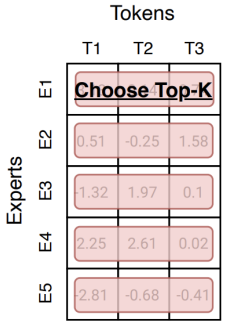
- Routing function
- Expert sizes
- Training objectives

ROUTING FUNCTION - OVERVIEW

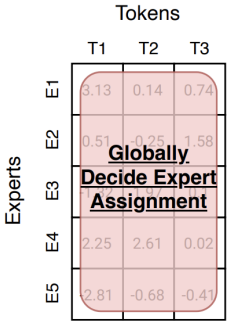
- Many of the routing algorithms boil down to 'choose top k'



Token chooses expert



Expert chooses token

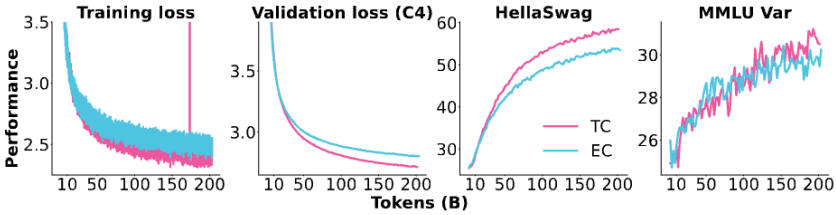


Global routing via optimization

[Fedus et al 2022]

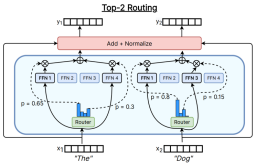
ROUTING TYPE

- Almost all the MoEs do a standard 'token choice topk' routing. Some recent ablations



COMMON ROUTING VARIANTS IN DETAIL

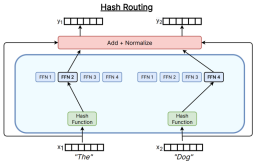
Top-k



Used in *most* MoEs

Switch Transformer (k=1)
Gshard (k=2), Grok (2), Mixtral (2), Qwen (4), DBRX (4), DeepSeek (7)

Hashing

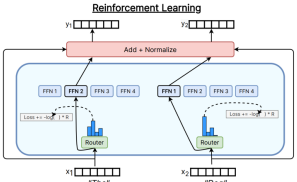


Common baseline

[Fedus et al 2022]

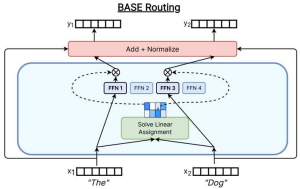
OTHER ROUTING METHODS

RL to learn routes



Used in some of the earliest work Bengio 2013, not common now

Solve a matching problem



Linear assignment for routing
Used in various papers like Clark '22

[Fedus et al 2022]

TOP-K ROUTING IN DETAIL.

- Most papers do the old and classic top-k routing. How does this work?
  - This is the DeepSeek (V1-2) router (Grok, Qwen do this too)
  - Mixtral, DBRX, DeepSeek v3 softmaxes after the TopK
- Gates selected by a logistic regressor

Gating

$$\mathbf{h}_t^l = \sum_{i=1}^{N} \left(g_{i,t} \, \mathrm{FFN}_i \left(\mathbf{u}_t^l \right) \right) + \mathbf{u}_t^l,$$

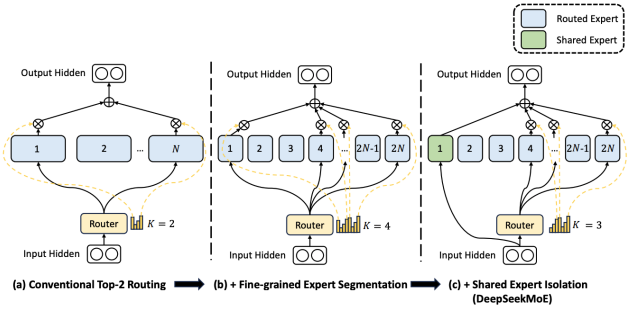
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i \left(\mathbf{u}_t^{l^T} \mathbf{e}_i^l \right),$$

[Dai et al 2024]

RECENT VARIATIONS FROM DEEPSEEK AND OTHER CHINESE LMS

- Smaller, larger number of experts + a few shared experts that are always on.
  - (Used in DeepSeek / Qwen, originally from DeepSpeed MoE)



VARIOUS ABLATIONS FROM THE DEEPSEEK PAPER

- More experts, shared experts all seem to generally help

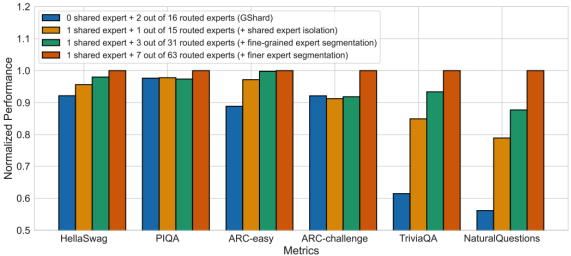
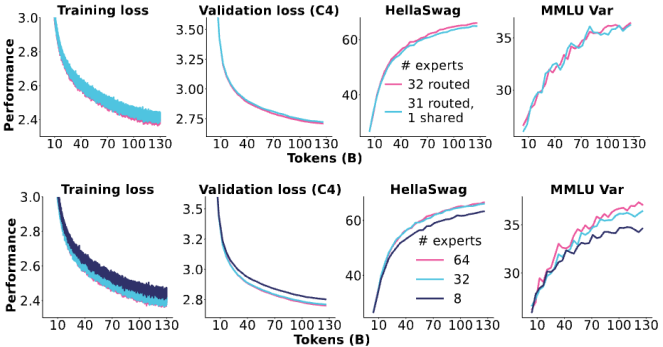


Figure 3 | Ablation studies for DeepSeekMoE. The performance is normalized by the best performance for clarity in presentation. All compared models have the same number of parameters and activated parameters. We can find that fine-grained expert segmentation and shared expert isolation both contribute to stronger overall performance.

ABLATIONS FROM OLMOE

- Gains from fine-grained experts, none from shared experts.



EXPERT ROUTING SETUPS FOR RECENT MOES

Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
ST-MOE	64	2	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OlMoE	64	8	0	1/8
MiniMax	32	2	0	$\sim$1/4
Llama 4 (maverick)	128	1	1	1/2

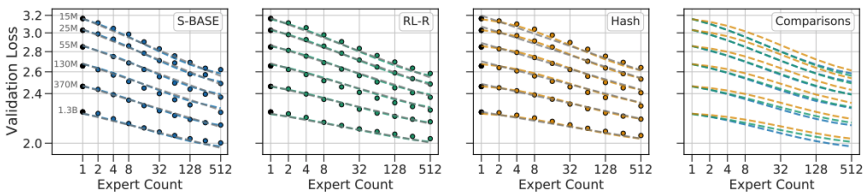
Training MoEs

HOW DO WE TRAIN MOES?

- Major challenge: we need sparsity for training-time efficiency...
  - But sparse gating decisions are not differentiable!
- Solutions?
- 1. Reinforcment learning to optimize gating policies 2. Stochastic perturbations 3. Heuristic 'balancing' losses.
  - Guess which one people use in practice?

RL FOR MOES

- RL via REINFORCE does work, but not so much better that it's a clear win
  - RL is the 'right solution' but gradient variances and complexity
  - means it's not widely used



(REINFORCE baseline approach, Clark et al 2020)

STOCHASTIC APPROXIMATIONS

- From Shazeer et al 2017 – routing decisions are stochastic with gaussian perturbations.
- 1. This naturally leads to experts that are a bit more robust. 2. The softmax means that the model learns how to rank K experts

$$G(x) = Softmax(KeepTopK(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i)$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

STOCHASTIC APPROXIMATIONS

- Stochastic jitter in Fedus et al 2022. This does a uniform multiplicative perturbation for the
- same goal of getting less brittle experts. This was later removed in Zoph et al 2022

```
if is_training:
    # Add noise for exploration across experts.
    router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

# Convert input to softmax operation from bfloat16 to float32 for stability.
router_logits = mtf.to_float32(router_logits)

# Probabilities for each token of what expert it should be sent to.
router_probs = mtf.softmax(router_logits, axis=-1)
```

Method	Fraction Stable	Quality (↑)
Baseline	4/6	**-1.755** ±0.02
Input jitter (10^{-2})	3/3	-1.777 ±0.03
Dropout (0.1)	3/3	-1.822 ±0.11

HEURISTIC BALANCING LOSSES

- Another key issue – systems efficiency requires that we use experts evenly..

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{4}$$

where f_i is the fraction of tokens dispatched to expert i,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \tag{5}$$

and P_i is the fraction of the router probability allocated for expert i, [2]

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \tag{6}$$

From the Switch Transformer [Fedus et al 2022]

The derivative with respect to $p_i(x)$ is $\frac{\alpha N}{T^2} \sum \mathbb{1}_{\text{argmax } p(x)=i}$, so more frequent use = stronger downweighting

EXAMPLE FROM DEEPSEEK (V1-2)

- Per-expert balancing – same as the switch transformer
- Per-device balancing – the objective above, but aggregated by device.

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i, \tag{12}$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i), \tag{13}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}, \tag{14}$$

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^{D} f'_i P'_i, \tag{15}$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \tag{16}$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j, \tag{17}$$

DEEPSEEK V3 VARIATION – PER-EXPERT BIASES

- Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

- They call this '**auxiliary loss free balancing**'

Complementary Sequence-Wise Auxiliary Loss. Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \tag{17}$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}\left(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r)\right), \tag{18}$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \tag{19}$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}, \tag{20}$$

(but the approach is not fully aux loss free..)

WHAT HAPPENS WHEN REMOVING LOAD BALANCING LOSSES?

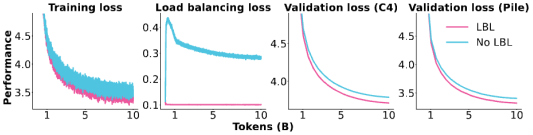


Figure 9: **Impact of applying a load balancing loss (LBL).** The training loss plot excludes the load balancing loss for both models. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

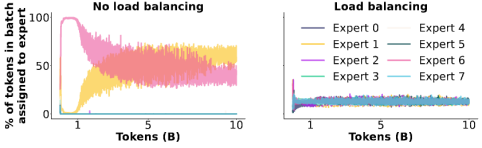
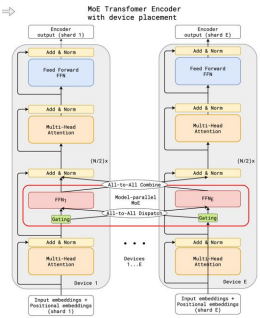


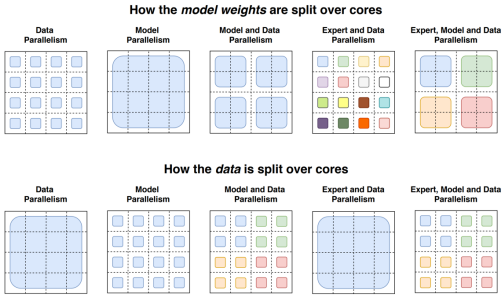
Figure 10: **Expert assignment during training when using or not using a load balancing loss for the first MoE layer.** More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo4OTkyNDg4

TRAINING MOES – THE SYSTEMS SIDE

MoEs parallelize nicely – Each FFN can fit in a device

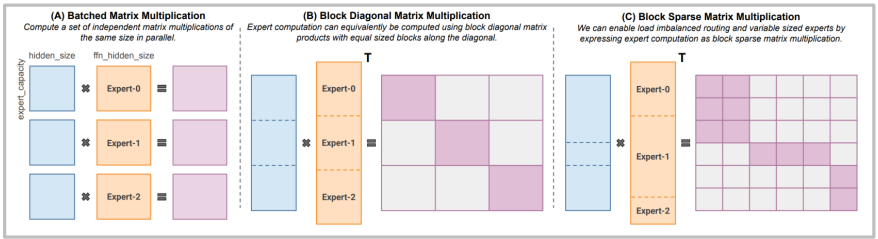


Enables additional kinds of parallelism



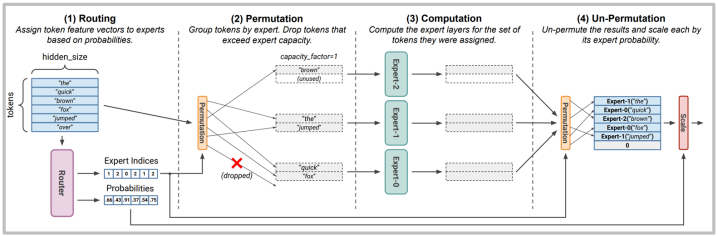
TRAINING MOES – THE SYSTEMS SIDE

- MoE routing allows for parallelism, but also some complexities
- Modern libraries like MegaBlocks (used in many open MoEs) use smarter sparse MMs

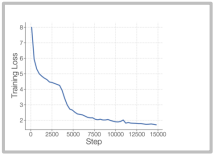


FUN SIDE ISSUE – STOCHASTICITY OF MOE MODELS

- There was speculation that GPT-4's stochasticity was due to MoE..
- Why would a MoE have additional randomness?
  - Token dropping from routing happens at a batch level – this means that
  - other people's queries can drop your token!



ISSUES WITH MOES - STABILITY



[7]Exponential functions have the property that a small input perturbation can lead to a large difference in the output. As an example, consider inputting 10 logits to a softmax function with values of 128 and one logit with a value 128.5. A roundoff error of 0.5 in `bfloat16` will alter the softmax output by 36% and incorrectly make all logits equal. The calculation goes from $\frac{\exp(0)}{\exp(0)+10\cdot\exp(-0.5)} \approx 0.142$ to $\frac{\exp(0)}{\exp(0)+10\cdot\exp(0)} \approx 0.091$. This occurs because the max is subtracted from all logits (for numerical stability) in softmax operations and the roundoff error changes the number from 128.5 to 128. This example was in `bfloat16`, but analogous situations occur in `float32` with larger logit values.

[Zoph 2022]

- **Solution:** Use Float 32 just for the expert router (sometimes with an aux z-loss)

$$L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^2 \tag{5}$$

Z-LOSS STABILITY FOR THE ROUTER

- What happens when we remove the z-loss?

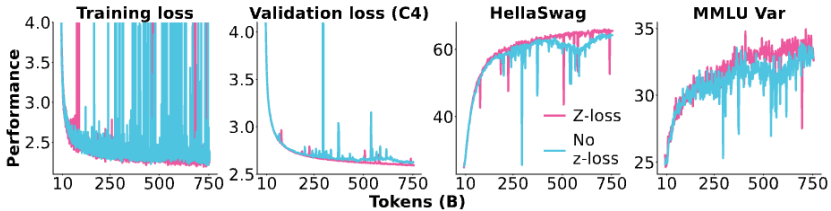
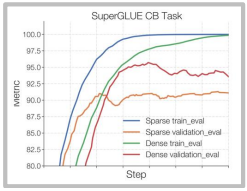


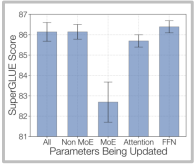
Figure 11: **Router z-loss.** We compare adding router z-loss with a loss weight of 0.001 versus no additional z-loss. More results, logs, and configurations: https://wandb.ai/ai2-llm/olmoe/reports/Plot-Zloss-vs-none--Vmlldzo4NDM4NjUz

ISSUES WITH MOES – FINE-TUNING

Sparse MoEs can overfit on smaller fine-tuning data



Zoph et al solution – finetune non-MoE MLPs



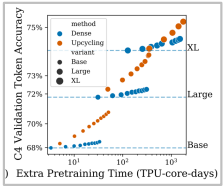
DeepSeek solution – use lots of data 1.4M SFT

Training Data. For training the chat model, we conduct supervised fine-tuning (SFT) on our in-house curated data, comprising 1.4M training examples. This dataset spans a broad range of categories including math, code, writing, question answering, reasoning, summarization, and more. The majority of our SFT training data is in English and Chinese, rendering the chat model versatile and applicable in bilingual scenarios.

Upcycling and DeepSeek MoE

OTHER TRAINING METHODS - UPCYCLING

- Can we use a pre-trained LM to initialize a MoE?



UPCYCLING EXAMPLE - MINICPM

- Uses the MiniCPM model (topk=2, 8 experts, ~ 4B active params).
- Simple MoE, shows gains from the base model with ~ 520B tokens for training

Model	C-Eval	CMMLU	MMLU	HumanEval	MBPP	GSM8K	MATH	BBH
Llama2-34B	-	-	62.6	22.6	33.0†	42.2	6.24	**44.1**
Deepseek-MoE (16B)	40.6	42.5	45.0	26.8	39.2	18.8	4.3	-
Mistral-7B	46.12	42.96	**62.69**	27.44	45.20	33.13	5.0	41.06
Gemma-7B	42.57	44.20	60.83	38.41	50.12	47.31	6.18	39.19
MiniCPM-2.4B	51.13	51.07	53.46	50.00	47.31	53.83	10.24	36.87
MiniCPM-MoE (13.6B)	**58.11**	**58.80**	58.90	**56.71**	**51.05**	**61.56**	**10.52**	39.22

Table 6: Benchmark results of MiniCPM-MoE. † means evaluation results on the full set of MBPP, instead of the hand-verified set (Austin et al., 2021). The evaluation results of Llama2-34B and Qwen1.5-7B are taken from their technical reports.

UPCYCLING EXAMPLE – QWEN MOE

- Qwen MoE – Initialized from the Qwen 1.8B model top-k=4, 60 experts w/ 4 shared.
- Similar architecture / setup to DeepSeekMoE, but one of the first (confirmed) upcycling successes

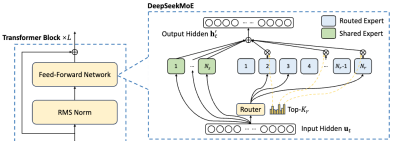
Model	#Parameters	#(Activated) Parameters	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	7.2	7.2	64.1	47.5	27.4	40.0	7.60
Qwen1.5-7B	7.7	7.7	64.6	50.9	32.3	-	-
Gemma-7B	8.5	7.8	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	16.4	2.8	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	14.3	2.7	62.5	61.5	34.2	40.8	7.17

DEEPSEEK MOE V1-V2-V3

To wrap up, we'll walk through the DeepSeek MoE architecture.

V1 (16B – 2.8 active):

Shared (2) + Fine-grained (64/4) experts



Standard, top-k routing

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

$$s_{i,t} = \mathrm{Softmax}_i(\mathbf{u}_t{}^T \mathbf{e}_i),$$

Standard Aux-loss balancing (Expert + Device)

$$\mathcal{L}_{\mathrm{ExpBal}} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i,$$

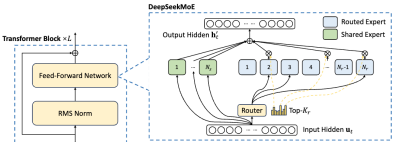
$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t},$$

DEEPSEEK MOE V2

V2 (236B – 21 active):

Shared (2) + Fine-grained (160/10) experts, 6 active



New things:

Top-M device routing

For DeepSeek-V2, beyond the naive top-K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most M devices. To be specific, for each token, we first select M devices that have experts with the highest affinity scores in them. Then, we perform top-K selection among experts on these M devices. In practice, we find that when $M \geqslant 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top-K routing.

Communication balancing loss – balancing both communication in *and* out

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^{D} f_i' P_i', \tag{29}$$

$$f_i' = \frac{D}{MT} \sum_{t=1}^{T} \mathbb{1}(\text{Token } t \text{ is sent to Device } i), \tag{30}$$

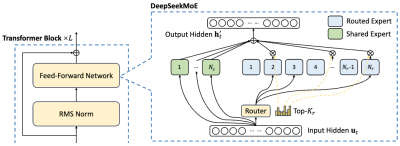
$$P_i' = \sum_{j \in \mathcal{E}_i} P_j, \tag{31}$$

where α_3 is a hyper-parameter called communication balance factor. The device-limited routing mechanism operates on the principle of ensuring that each device transmits at most MT hidden states to other devices. Simultaneously, the communication balance loss is employed to encourage each device to receive around MT hidden states from other devices. The communication balance loss guarantees a balanced exchange of information among devices, promoting efficient communications.

DEEPSEEK MOE V3

V3 (671B – 37 active):

Shared (1) + Fine-grained (258) experts, 8 active



New things

Sigmoid+Softmax topK + topM

$$\mathbf{h}_t' = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t}\,\mathrm{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$

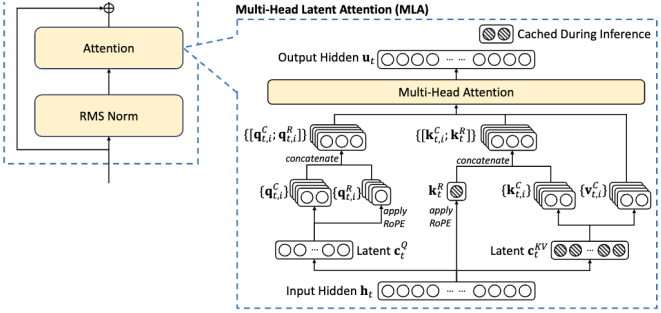
$$s_{i,t} = \mathrm{Sigmoid}\left(\mathbf{u}_t^T \mathbf{e}_i\right),$$

Aux-loss-free + seq-wise aux

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}. \end{cases}$$

BONUS: WHAT ELSE DO YOU NEED TO MAKE DEEPSEEK MOE V3?

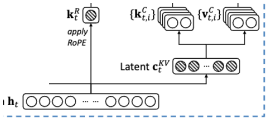
- MLA : Multihead, latent attention
- Basic idea: express the Q, K, V as functions of a lower-dim, 'latent' activation



WHAT ELSE DO YOU NEED TO MAKE DEEPSEEK MOE V3?

Basic idea: express the Q, K, V as functions of a lower-dim, 'latent' activation

$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t,$$
$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV},$$
$$\mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV},$$



Benefits: when KV-caching, we only need to store c_t^{KV}, which can be much smaller. W^{UK} can be merged into the Q projection

(they also compress queries, for memory savings during training) $\mathbf{c}_t^Q = W^{DQ}\mathbf{h}_t,$ $\mathbf{q}_t^C = W^{UQ}\mathbf{c}_t^Q,$

Complexity: rope conflicts with MLA-style caching.

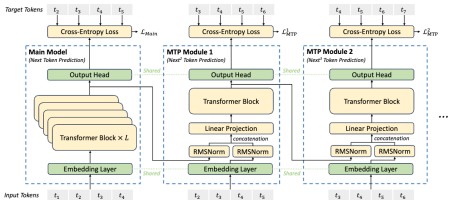
Without RoPE – $\langle Q, K\rangle = \langle hW^Q, W^{UK}c_t^{KV}\rangle = \langle h\,W^QW^{UK}, c_t^{KV}\rangle$

With RoPE – $\langle QR_q, R_kK\rangle = \langle hW^QR_q, R_kW^{UK}c_t^{KV}\rangle = \langle h\,W^QR_qR_kW^{UK}, c_t^{KV}\rangle$

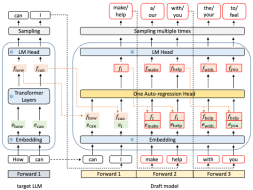
The solution – Have a few non-latent key dimensions that can be rotated

WHAT ELSE DO YOU NEED TO MAKE DEEPSEEK MOE V3?

MTP: Have small, lightweight models that predict multiple steps ahead



(But they only do MTP with one token ahead)
[Deepseek v3]



[EAGLE]

$$\mathbf{h}_i'^k = M_k[\mathrm{RMSNorm}(\mathbf{h}_i^{k-1}); \mathrm{RMSNorm}(\mathrm{Emb}(t_{i+k}))],$$
$$\mathbf{h}_{1:T-k}^k = \mathrm{TRM}_k(\mathbf{h}_{1:T-k}'^k),$$
$$P_{i+k+1}^k = \mathrm{OutHead}(\mathbf{h}_i^k).$$

(See paper for ablations)

MOE SUMMARY

- MoEs take advantage of sparsity – not all inputs need the full model
  - Discrete routing is hard, but top-k heuristics seem to work
- Lots of empirical evidence now that MoEs work, and are cost-effective

Where This Is Going

WHERE THIS IS GOING

- The changes in this session act on the Transformer *block*, not on the type of data.
- Vision Transformers use them too, e.g.:
  - ViT-22B: parallel layers, QK normalization, no biases on the QKV projections and LayerNorms ▸ Dehghani et al., 2023;
  - V-MoE: a sparse mixture-of-experts Vision Transformer ▸ Riquelme et al., 2021.
- Session 10 takes the Transformer to images, multimodal data and video.

License & Attribution

LICENSE & ATTRIBUTION

- **Source.** All frames except the four listed below are adapted from **Stanford CS336 *Language Modeling from Scratch***, Spring 2025, Lectures 3 (*Architectures, hyperparameters*) and 4 (*Mixture of experts*) by Tatsunori Hashimoto, `https://github.com/stanford-cs336/spring2025-lectures` — **Copyright 2025 Stanford University**, licensed under the **MIT License** (full notice in `_sources/`). One frame per source slide, in the source's order; text and formulas retypeset in the source's wording, figures cropped from the source PDF.
- **Figures** inside the slides are from the papers credited on each frame, reused with that attribution.
- **Original to this course.** "Since 2017: What Changed Inside the Transformer?", "Compute vs. Memory", "The KV Cache", "Where This Is Going".